

Compare Apriori and FP-Growth Algorithms on a Large Transactional Dataset

1. Introduction

Association Rule Mining discovers relationships among items in transactional data. This project compares Apriori and FP-Growth algorithms based on frequent itemsets and execution time to identify the more efficient approach for large datasets.

2. Architecture Diagram / Flowchart

Transaction Dataset -> Data Preprocessing -> One-Hot Encoding -> Apriori & FP-Growth -> Performance Comparison -> Results

3. Implementation

Tools and Technologies Used

Python, Google Colab, Pandas, mlxtend, Time module
Algorithms: Apriori and FP-Growth

Code

```
!pip install -q mlxtend
```

```
import pandas as pd
```

```
import time
```

```
from mlxtend.preprocessing import TransactionEncoder
```

```
from mlxtend.frequent_patterns import apriori, fpgrowth
```

```
# Sample Transaction Dataset
```

```
transactions = [
```

```
    ['Milk', 'Bread', 'Butter'],
```

```
    ['Bread', 'Butter'],
```

```
    ['Milk', 'Bread'],
```

```
    ['Milk', 'Butter'],
```

```
    ['Bread', 'Eggs'],
```

```
['Milk','Bread','Butter','Eggs'],  
['Milk','Eggs'],  
['Bread','Butter','Jam'],  
['Milk','Bread','Jam'],  
['Butter','Jam'],  
['Milk','Bread','Butter'],  
['Milk','Butter'],  
['Bread','Butter','Eggs'],  
['Milk','Bread','Eggs'],  
['Milk','Bread','Butter','Jam']  
]
```

```
# Convert to One-Hot Encoding
```

```
te = TransactionEncoder()
```

```
te_array = te.fit(transactions).transform(transactions)
```

```
df = pd.DataFrame(te_array, columns=te.columns_)
```

```
# Apriori
```

```
start = time.time()
```

```
apriori_result = apriori(df, min_support=0.2, use_colnames=True)
```

```
apriori_time = time.time() - start
```

```
# FP-Growth
```

```
start = time.time()
```

```
fpgrowth_result = fpgrowth(df, min_support=0.2, use_colnames=True)
```

```
fpgrowth_time = time.time() - start
```

```
print("Apriori Frequent Itemsets")
```

```
print(apriori_result)
```

```
print("\nFP-Growth Frequent Itemsets")
```

```
print(fpgrowth_result)
```

```
print("\nExecution Time Comparison")
```

```
print("Apriori Time:", apriori_time)
```

```
print("FP-Growth Time:", fpgrowth_time)
```

```
if apriori_time > fpgrowth_time:
```

```
    print("\nFP-Growth is Faster")
```

```
else:
```

```
    print("\nApriori is Faster")
```

4. Results

Output Screenshot

```
... Apriori Frequent Itemsets
      support      itemsets
0    0.733333      (Bread)
1    0.666667      (Butter)
2    0.333333      (Eggs)
3    0.266667      (Jam)
4    0.666667      (Milk)
5    0.466667    (Butter, Bread)
6    0.266667    (Eggs, Bread)
7    0.200000    (Jam, Bread)
8    0.466667    (Milk, Bread)
9    0.200000    (Butter, Jam)
10   0.400000    (Butter, Milk)
11   0.200000    (Milk, Eggs)
12   0.266667 (Butter, Milk, Bread)

FP-Growth Frequent Itemsets
      support      itemsets
0    0.733333      (Bread)
1    0.666667      (Milk)
2    0.666667      (Butter)
3    0.333333      (Eggs)
4    0.266667      (Jam)
5    0.466667    (Milk, Bread)
6    0.466667    (Butter, Bread)
7    0.400000    (Butter, Milk)
8    0.266667 (Butter, Milk, Bread)
9    0.266667    (Eggs, Bread)
10   0.200000    (Milk, Eggs)
```

Terminal

Explanation of Results

Both algorithms generate frequent itemsets from the same dataset. FP-Growth generally executes faster because it avoids candidate generation, making it more suitable for large transactional datasets.

Runnable Project Link

<https://colab.research.google.com/drive/1t5U-tfaHrjG9tBwiMVi--vSzi01WtdXl#scrollTo=6otRjEYW2Dxv>

Github Link: <https://github.com/ommalewar8154-wq/Apriori-vs-FP-Growth-Comparison>

5. Viva Questions

What is Apriori?

Answer: Association rule mining algorithm using candidate generation.

What is FP-Growth?

Answer: Frequent pattern mining using an FP-Tree.

Which algorithm is faster?

Answer: FP-Growth.

What is minimum support?

Answer: Minimum frequency for an itemset to be frequent.

Where are these algorithms used?

Answer: Market basket analysis and recommendation systems.